Coinductive Homomorphism Reinforcement Learning

Optimality as Structure Preservation

Ricardo Corral-Corral

April 2026

Abstract

Traditional reinforcement learning defines optimality by maximizing a scalar return—a criterion that collapses infinite reward sequences into a single number, discarding their temporal structure. We propose a different foundation: an action is optimal when it leads to a state from which the agent's future is best.

What makes one state better than another? Acting optimally from it yields higher rewards at every future depth. And what makes *those* rewards higher? The states reachable from there are themselves better—by the same criterion, unfolding without bound. This coinductive structure is the core of our definition: a **coinductive homomorphism** is an action ranking that preserves the quality ordering on the states actions lead to.

The definition is an order homomorphism from action rankings into a coinductively ordered codomain, and it decouples optimality from immediate transition rewards. An action that sacrifices immediate reward to reach a superior state is correctly ranked—a pattern that arises naturally in investment, exploration, and preparation.

A natural strengthening additionally requires that immediate transition rewards respect the ranking, bundling two orthogonal conditions into a single requirement on the full action-value stream. We prove that the two conditions decompose cleanly, that the coinductive homomorphism alone is strictly more general, and that the top-ranked action maximizes cumulative return from successor states at every finite horizon. Three verified environments of increasing structural complexity demonstrate settings where the coinductive homomorphism correctly ranks actions but its action-value strengthening cannot.

We prove seven main results, all machine-verified in Agda: **(1)** Subsumption: both conditions imply classical argmax optimality for any finite horizon; **(2)** Monotonic learning: ranking updates via violation-correction converge in at most $|S| \cdot \binom{|A|}{2}$ swaps; **(3)** Instant adaptation: when actions become unavailable, the next-best action is $O(1)$ lookup; **(4)** Stochastic extension: the framework lifts to probabilistic environments via the Giry monad, where rankings preserve lexicographic expected stream dominance; **(5)** Distributional hierarchy: FOSD and the SD[k] chain give parameterized verification strength for broader utility classes; **(6)** Compositional algebra: verified rankings compose via product, sum, convolution, and scaling; **(7)** State abstraction: rankings verified on finite abstract systems lift to arbitrary concrete state spaces; **(8)** Scalable verification: a modular Environment Class architecture provides automatic trace-to-stream bridges, demonstrated with a full coinductive homomorphism for 8-Queens (8^8 search space, zero postulates).

This document is itself the proof: compiling it with `agda --latex` type-checks all embedded code while producing this PDF.

1 Motivation: The Limitations of Scalar Returns

Reinforcement learning (RL) studies agents that learn to make sequential decisions by interacting with an environment [1]. At each discrete time step, the agent observes a *state*, selects an *action*, and receives a numerical *reward* that signals how good the outcome was. The agent's goal is to find a *policy*—a rule mapping states to actions—that collects as much reward as possible over time.

The question is how to define "as much reward as possible." The dominant answer in modern RL is the *expected discounted return*: given a policy π, the agent averages over all trajectories τ it might experience:

$$J(\pi) = \mathbb{E}_{\tau\sim\pi}\left[\sum_{t=0}^{\infty} \gamma^t r_t\right]$$

where r_t is the reward at time t and $\gamma \in [0, 1)$ is the *discount factor*. The discount makes future rewards exponentially less valuable: a reward 10 steps ahead counts for only γ^{10} of its face value. This ensures the sum converges and makes the problem mathematically tractable.

The result is a single scalar. A policy is optimal when it maximizes this number. This formulation—rooted in Bellman's dynamic programming principle [16] and operationalized by algorithms like Q-learning [17], policy gradient methods, and deep RL [18]—has driven remarkable successes, from game playing to robotics.

1.1 The Problem: Temporal Structure Loss

The difficulty is that this scalar discards the *temporal structure* of the reward sequence. Consider two futures:

Future	**Reward Sequence**	**Sum (γ=0.9)**
A: "Win then die"	$[10, -5, 0, 0, \ldots]$	$10 - 4.5 = 5.5$
B: "Struggle then thrive"	$[0, 5, 0, 0, \ldots]$	$0 + 4.5 = 4.5$

A scalar optimizer chooses Future A. But *structurally*, Future B may be preferable—it avoids catastrophic failure and enters a productive state. The sum erases this distinction. This is **value aliasing**: structurally different futures mapping to the same scalar.

The problem deepens with several well-known artifacts of scalar aggregation:

- **The Discount Factor (γ):** An extrinsic parameter that controls how quickly future rewards are attenuated. It systematically undervalues delayed payoffs, making the agent short-sighted by design. No single γ is correct for all environments, yet the optimal policy can change discontinuously as γ varies.
- **Instability:** Q-learning bootstraps scalar estimates of $J(\pi)$, leading to the deadly triad of function approximation, bootstrapping, and off-policy learning [1].
- **Credit Assignment:** A reward at t=1000 is discounted by $\gamma^{1000} \approx 0$, making long-horizon learning effectively impossible under standard discounting.

These are not implementation bugs—they are consequences of the scalar return formulation itself. When the best course of action requires accepting a cost now in exchange for a benefit later, discounting can make the sacrifice appear irrational.

1.2 Our Approach

We propose to define optimality via *structure preservation* rather than scalar maximization. Instead of compressing the reward sequence into a number, we keep it as an infinite stream and define action rankings that preserve the quality ordering of successor states at every depth. The question is: what structure should be preserved, and how?

2 Structural Optimality: Successor State Quality

Consider an agent choosing between two actions at a state. Action a gives an immediate reward of 3 but leads to a dead-end state (reward 0 forever). Action b gives immediate reward 0 but leads to a productive state (reward 5 forever).

A scalar return optimizer with low discount factor might prefer a. But structurally, b is the better action—it leads to a *better state*. The immediate reward is a one-time cost; the successor state quality compounds forever.

This motivates our central criterion:

An action is ranked higher because it leads to a successor state of higher quality. A state is of higher quality when acting optimally from it yields higher rewards at every future depth—which itself depends on the quality of states reachable from there, unfolding coinductively.

The ranking tracks which actions lead to *success*—both in the sense of successor state and in the evaluative sense.

3 Formal Definition

3.1 The Order Homomorphism

We now make the intuition from Section 2 precise. The formalization requires three ingredients; we introduce each in turn.

Actions and rankings. Let $\mathcal{A}$ be the finite set of actions available to the agent. At each state s, the agent maintains a *ranking*—a total preorder $\preceq_a$ on $\mathcal{A}$—expressing which actions it considers at least as good as which others. If $a \preceq_a b$, the agent regards b as at least as good as a at state s.

Successor states. When the agent takes action a at state s, the environment transitions to a new state and emits a reward. We write $\mathit{step}(s, a)$ for this transition, which produces a pair: the successor state $\mathit{next}(s, a)$ and the immediate reward $r(s, a)$. The successor state is where the agent *ends up*—the starting point of the rest of its life.

Value streams. From any state s', the agent can act optimally into the indefinite future, collecting a reward at each time step. Rather than collapsing this sequence into a scalar, we keep it as an infinite stream:

$$\mathit{value}(s') = \left[\, v_0,\ v_1,\ v_2,\ \ldots \right]$$

where v_t is the optimal reward achievable at depth t from s'. This stream retains the full temporal structure that a scalar return discards. (The precise construction is given in §4.)

Comparing streams. To say one state is "better" than another, we compare their value streams *pointwise*. We write $x \preceq_s y$ when stream x is dominated by stream y at every position: for all $t = 0, 1, 2, \ldots$, the t-th element of x is $\preceq_r$ the t-th element of y. This is a strong condition: the better stream must be at least as good at every single future depth, not just on average or in total.

With these ingredients in hand, the definition is a single sentence:

Definition 1 (Coinductive Homomorphism)**.** The ranking $\preceq_a$ is a **coinductive homomorphism** at state s if:

$$a \preceq_a b \implies \mathit{value}(\mathit{next}(s, a)) \preceq_s \mathit{value}(\mathit{next}(s, b))$$

In words: if the ranking says b is at least as good as a, then the state reached by b must have a value stream that dominates the value stream of the state reached by a—at every future depth, without exception.

The definition says nothing about the immediate rewards $r(s, a)$ and $r(s, b)$. It judges actions entirely by *where they lead*. An action that pays a cost now but reaches a superior state is correctly ranked, because the successor state's value stream—reflecting the entire optimal future from that point onward—is what matters.

Why "homomorphism"? The map $a \mapsto \mathit{value}(\mathit{next}(s, a))$ sends each action to a stream. The definition requires this map to be *order-preserving*: if $a \leq_a b$ in the action ranking, then $\mathit{value}(\mathit{next}(s, a)) \leq_s \mathit{value}(\mathit{next}(s, b))$ in the stream ordering. An order-preserving map between ordered sets is called a *homomorphism*—it preserves the structure. The qualifier "coinductive" refers to the target: the stream ordering $\leq_s$ unfolds to arbitrary depth, verified one position at a time, without bound—an infinite tower of commuting diagrams:

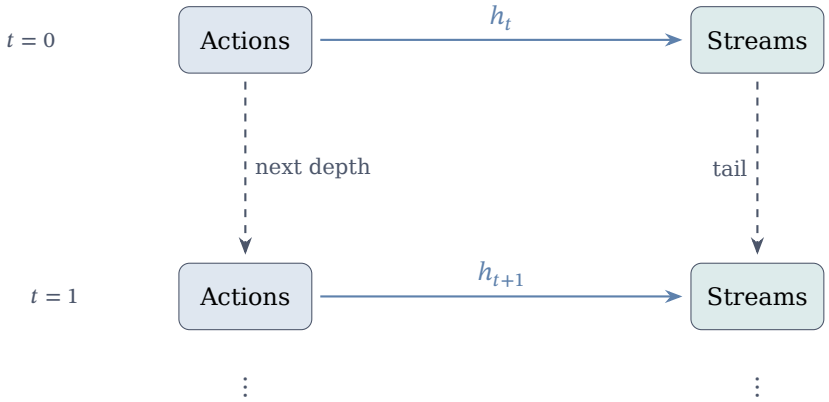


Figure 1: The coinductive tower: h_t maps each action to its successor-value stream at depth t. Ranking preservation ($a \leq_a b \Rightarrow h_t(a) \leq_s h_t(b)$) must hold at every level, verified by unfolding one `tail` at a time.

This is a **strategic** evaluation of actions: it assesses the absolute potential of the successor state, allowing for immediate tactical sacrifices.

3.2 The Action-Value Specialization

The coinductive homomorphism evaluates successor state quality—the agent's future *after* the transition. A natural strengthening evaluates the *entire* consequence of an action, including the immediate reward it produces.

Given action a at state s, define the *action-value stream*:

$$\mathit{action\text{-}value}(s, a) = \left[\, r(s, a),\ \ v_0,\ \ v_1,\ \ v_2,\ \ \ldots \,\right]$$

Its head is the immediate transition reward; its tail is the value stream of the successor state. The action-value stream is a richer object: it records both what the action pays now and what it sets up for the future.

Definition 2 (Action-Value Coinductive Homomorphism)**.** The ranking $\leq_a$ is an **action-value coinductive homomorphism** at state s if:

$$a \leq_a b \implies \mathit{action\text{-}value}(s, a) \leq_s \mathit{action\text{-}value}(s, b)$$

Because $\leq_s$ compares streams pointwise, this bundles *two* requirements into one: the immediate rewards must compare correctly (the head), *and* the successor state qualities must compare correctly (the tail).

This is a **tactical** evaluation: it demands that the better-ranked action wins both the immediate payoff and the long-term future. When both conditions hold, the ranking is unassailable. But when the optimal action sacrifices immediate reward for a superior successor—the pattern from Section 2—the head condition breaks, and this definition cannot express the correct ranking.

3.3 Decomposition

The relationship between the two definitions is precise:

Theorem 1 (Decomposition)**.** *The action-value condition decomposes into two independent conditions:*

1. ***Successor quality*** *(CoinductiveHomomorphism):* $a \leq_a b \implies value(next(s,a)) \leq_s value(next(s,b))$
2. ***Immediate reward compatibility****:* $a \leq_a b \implies r(s,a) \leq_r r(s,b)$

Either condition can hold independently. Together, they are equivalent to the action-value condition.

The first condition is the coinductive homomorphism—the strategic evaluation. The second is a purely local constraint on immediate rewards. The action-value condition is their conjunction: it insists on tactical *and* strategic soundness simultaneously.

When immediate rewards are aligned with the ranking—as they are in environments with uniform step costs, shaped rewards, or goal-terminal structure—the second condition holds for free, and the two definitions coincide. The gap opens only in environments where the optimal action has a worse immediate reward than a suboptimal alternative: the sacrifice pattern.

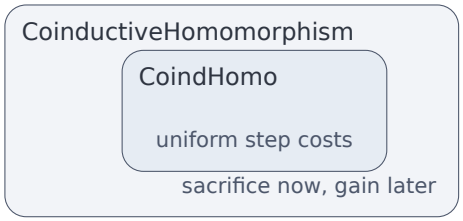


Figure 2: CoindHomo is strictly contained in CoinductiveHomomorphism. Environments with uniform step costs satisfy both. Environments requiring immediate reward sacrifice separate them.

4 The Agda Core

We now present the verified implementation. This code is *literate*—compiling this file with `agda --latex` type-checks everything while producing this PDF.

```
{-# OPTIONS --safe --guardedness #-}

module CSHRL-Successor where

open import Data.List using (List; map; foldr; []; _∷_)
open import Codata.Guarded.Stream using (Stream; head; tail; _∷_; tabulate)
open import Relation.Binary.PropositionalEquality using (_≡_; _≢_; refl)
open import Data.Product using (proj₁; proj₂; _×_; _,_)
open import Relation.Nullary using (¬_; Dec; yes; no)
open import Function using (_∘_)
open import Data.Nat using (ℕ; zero; suc; _⊔_)
open import Data.Bool using (Bool; true; false; if_then_else_)
```

The `Core` module is parameterized by the environment:

```
module Core
  (State Action Reward : Set)
  (step        : State → Action → State × Reward)
  (_≤ᵣ_        : Reward → Reward → Set)
  (max         : Reward → Reward → Reward)
  (bottom      : Reward)
  (all-actions : List Action)
  where
```

```
infix 4 _≤ₛ_

StreamR : Set
StreamR = Stream Reward

max-list : List Reward → Reward
max-list = foldr max bottom
```

4.1 Value and Action-Value Streams

The key construction: `value` is an infinite stream obtained by tabulating finite-horizon solutions.

```
solve : State → ℕ → Reward
solve s zero = max-list (map (λ a → proj₂ (step s a)) all-actions)
solve s (suc n) = max-list (map (λ a → solve (proj₁ (step s a)) n) all-actions)

value : State → StreamR
value s = tabulate (solve s)

action-value : State → Action → StreamR
head (action-value s a) = proj₂ (step s a)
tail (action-value s a) = value (proj₁ (step s a))
```

`solve s n` computes the maximum reward achievable *exactly* n *steps from now*, independently maximizing the action at every intermediate step. Crucially, the action sequence that maximizes the reward at depth n may differ from the one that maximizes at depth m. The resulting value stream is therefore *not a trajectory*—no single action sequence produces it. It is the pointwise upper envelope of all possible trajectories: the agent's **capability profile** from state s, recording the best it *could* achieve at each future depth.

CoinductiveHomomorphism compares these capability profiles. When state s_b's profile dominates s_a's at every depth, the agent prefers any action that reaches s_b—even if that action sacrifices immediate reward. The sacrifice is invisible in the profile: it only records what the agent *can* reach, not the cost of getting there.

The `action-value` of action a at state s is a stream whose head is the immediate reward and whose tail is the capability profile of the successor state.

4.2 Coinductive Stream Ordering

```
record _≤ₛ_ (x y : StreamR) : Set where
  coinductive
  field
    head≤ : head x ≤ᵣ head y
    tail≤ : tail x ≤ₛ tail y

open _≤ₛ_ public
```

A proof of $x \leq_s y$ is an infinite witness: at every time step, the heads compare correctly.

4.3 Why Propositions Instead of Booleans?

The reward ordering `_≤ᵣ_` lives in `Set`—it is a *proposition*, not a Boolean decision procedure. This is a deliberate design choice:

- **Booleans are blind:** `true` carries no information about *why* the comparison holds. To use a Boolean result in a proof, a separate soundness lemma is required.

- **Propositions carry evidence:** A value of type `r₁ ≤ᵣ r₂` *is* the proof—no lemma needed.
- **Decidability bridges both:** `Dec (r₁ ≤ᵣ r₂)` is either `yes p` (with proof `p`) or `no ¬p` (with refutation).

This separation threads through the entire architecture: the Core module uses propositions for proofs; the Finder uses Booleans for speed; the Environment Class infrastructure bridges them via `Dec`.

4.4 CoindHomo: The Action-Value Condition

The `CoindHomo` record is the Agda encoding of the Action-Value Coinductive Homomorphism (Definition 2):

```
record CoindHomo : Set₁ where
  field
    _≤ₐ_ : State → Action → Action → Set
    preserves : ∀ a b s → _≤ₐ_ s a b →
                action-value s a ≤ₛ action-value s b

open CoindHomo {{...}} public
```

The record has two fields. The first, `_≤ₐ_`, is the ranking itself—a ternary relation indexed by state, taking two actions and returning a type (a proposition, per §4.3). The second, `preserves`, is the proof obligation: for any actions a, b and state s, if the ranking says $a \leq_a b$, then the full action-value stream of a is coinductively dominated by that of b. Because $\leq_s$ checks both head and tail, this bundles two requirements: the immediate reward must compare correctly (head), and the successor state quality must compare correctly (tail).

4.5 CoinductiveHomomorphism: The Successor Condition

The `CoinductiveHomomorphism` record is the Agda encoding of Definition 1. The successor map extracts the state reached by an action:

```
next : State → Action → State
next s a = proj₁ (step s a)

successor-value : State → Action → StreamR
successor-value s a = value (next s a)
```

`successor-value` $s\ a$ is the optimal value stream from the state reached by taking action a at state s. It is definitionally equal to *tail*(*action-value* $s\ a$).

```
record CoinductiveHomomorphism : Set₁ where
  field
    _≤ₐ_ : State → Action → Action → Set
    preserves : ∀ a b s → _≤ₐ_ s a b →
                successor-value s a ≤ₛ successor-value s b
```

CoinductiveHomomorphism requires only that the ranking preserves successor state quality. The immediate transition reward is decoupled by design.

4.6 The Decomposition

```
CoindHomo→CoinductiveHomomorphism : CoindHomo → CoinductiveHomomorphism
CoindHomo→CoinductiveHomomorphism homo = record
  { _≤ₐ_ = CoindHomo._≤ₐ_ homo
  ; preserves = λ a b s prf → tail≤ (CoindHomo.preserves homo a b s prf)
  }
```

Every CoindHomo is a CoinductiveHomomorphism: project the tail of the action-value stream dominance. The converse does *not* hold.

```
CoinductiveHomomorphism+Head→CoindHomo :
  (ch : CoinductiveHomomorphism) →
  (∀ a b s → CoinductiveHomomorphism._≤ₐ_ ch s a b →
    proj₂ (step s a) ≤ᵣ proj₂ (step s b)) →
  CoindHomo
CoinductiveHomomorphism+Head→CoindHomo ch head-compat = record
  { _≤ₐ_ = CoinductiveHomomorphism._≤ₐ_ ch
  ; preserves = λ a b s prf →
      combine-≤ₛ (head-compat a b s prf)
                 (CoinductiveHomomorphism.preserves ch a b s prf)
  }
```

When immediate transition rewards also respect the ranking, CoinductiveHomomorphism specializes to CoindHomo. This characterizes exactly what CoindHomo adds: the head constraint, which is orthogonal to successor state quality.

5 When the Conditions Coincide—and When They Diverge

The decomposition theorem reveals that CoindHomo and CoinductiveHomomorphism differ by exactly one condition: whether immediate transition rewards respect the ranking. When does this matter?

5.1 Aligned Environments: The Common Case

In a large class of environments, the head condition holds trivially—and the two definitions coincide. This class includes:

- **Uniform step costs.** If all actions at a given state yield the same immediate reward, the head condition reduces to $r \leq_r r$—reflexivity. Examples: FrozenLake (all non-terminal steps give 0), MountainCar (all steps give -1), navigation grids with uniform penalties.
- **Shaped rewards.** When reward functions are engineered so that immediate feedback points toward the optimal policy, the head condition is satisfied by construction. This is common in continuous-control benchmarks (Pendulum, LunarLander) where shaping rewards guide the agent toward desirable configurations.
- **Goal-terminal environments.** If the episode ends upon reaching a goal state and all non-terminal transitions have equal cost, the only reward distinction is reaching the goal versus not. The head condition holds at every non-terminal state.

For these environments, every CoinductiveHomomorphism is automatically a CoindHomo, and vice versa. Results proved under CoindHomo—the Finder algorithm, propagation theorems, environment class constructions, verified instantiations—apply directly and without modification. The environments are not "easier" in any pejorative sense; they are the standard decision problems that the reinforcement learning community has studied for decades, and they are well-served by either condition.

5.2 The Sacrifice Pattern: An Unexplored Realm

The gap between the two conditions opens when the optimal action has a *worse* immediate reward than a suboptimal alternative. We call this the **sacrifice pattern**: the agent must pay an immediate cost to reach a superior successor state.

This pattern is structurally interesting because it is precisely where discount factors also struggle. A discount factor $\gamma < 1$ exponentially suppresses future rewards; a sacrifice whose payoff arrives k steps later is scaled by γ^k. For deep sacrifices—multi-stage investment chains, extended preparation sequences—the discounted return of the sacrificing action can fall below the myopic alternative, even though the undiscounted future is strictly superior at every depth.

CoindHomo and discounted returns thus share a blind spot: both anchor optimality to the present. Discounting does so by geometric decay; CoindHomo does so via the head condition. CoinductiveHomomorphism is the first condition in this framework that fully decouples optimality from immediate reward, judging actions solely by the quality of states they lead to.

5.3 Why the Sacrifice Pattern Is Rare in Benchmarks

The observation that CoindHomo suffices for virtually every standard environment is not a coincidence. Reinforcement learning benchmarks have been developed in a culture where discounted scalar return is the optimality criterion. Reward functions are carefully shaped to *eliminate* the sacrifice pattern—that is literally the purpose of reward shaping. Environments where immediate rewards actively mislead are regarded as "hard exploration problems" and are typically studied as pathological cases rather than representative ones.

The result is a selection bias: the environments that survive into textbooks and standard libraries are precisely those where immediate rewards already point toward optimality. The sacrifice pattern—where the greedy action is wrong—has been quietly engineered out.

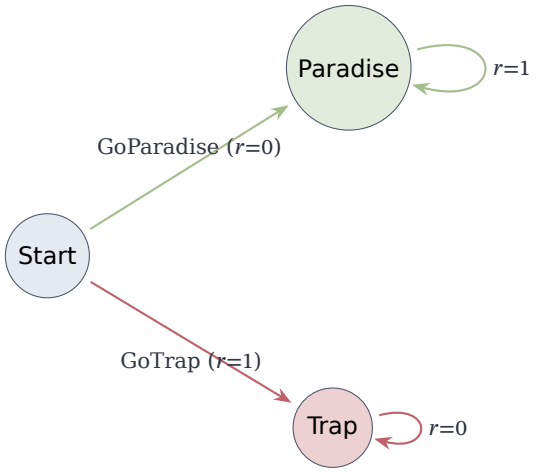
This does not diminish the value of CoindHomo. It is the right tool for aligned environments, and the vast majority of practical decision problems have this structure. Both conditions are introduced here as part of a unified framework: CoindHomo handles the common case cleanly, while CoinductiveHomomorphism extends the formal reach to environments where immediate rewards conflict with long-term quality—without losing anything that CoindHomo provides.

6 Strict Generality: Verified Counterexamples

The implication CoindHomo ⇒ CoinductiveHomomorphism is strict. We prove this with three verified environments of increasing structural complexity, each exhibiting the sacrifice pattern.

6.1 Sacrifice Now, Gain Later

The minimal separating example has three states and two actions:



GoParadise sacrifices the immediate reward (0 vs 1) but leads to Paradise (reward 1 forever). GoTrap grabs the immediate reward but leads to Trap (reward 0 forever).

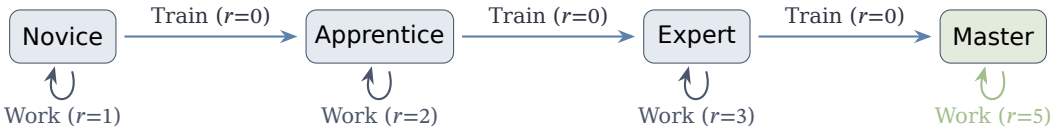
CoinductiveHomomorphism: $value(\text{Trap}) = [0, 0, \ldots] \leq_s [1, 1, \ldots] = value(\text{Paradise})$ □

CoindHomo: Cannot rank GoTrap ≤ GoParadise (head requires $1 \leq 0$). Cannot rank GoParadise ≤ GoTrap either (tail comparison yields $1 \leq 0$ at the head of the successor streams). The actions are *incomparable*—even though one is clearly better.

Full proof: `src/CSHRL/Tasks/Verified/BinarySacrifice.agda`

6.2 Skill Investment: Multi-Stage Sacrifice

A chain of investment decisions where training sacrifices immediate reward to advance:



The capability profiles (value streams) form a clean progression:

$$\begin{aligned} \text{Master:} &\quad [5, 5, 5, 5, \ldots] \\ \text{Expert:} &\quad [3, 5, 5, 5, \ldots] \\ \text{Apprentice:} &\quad [2, 3, 5, 5, \ldots] \\ \text{Novice:} &\quad [1, 2, 3, 5, \ldots] \end{aligned}$$

These are *not trajectories*. Consider the Apprentice profile $[2, 3, 5, \ldots]$: the 2 at depth 0 comes from Working (best immediate reward), the 3 at depth 1 comes from Training to Expert *then* Working (a different action plan), and the 5 at depth 2 comes from Training twice to Master then Working (yet another plan). Each entry independently records the best the agent *could* achieve at that depth. No single action sequence produces the stream $[2, 3, 5, \ldots]$.

CoinductiveHomomorphism compares these profiles: Train's successor (Expert, profile $[3, 5, 5, \ldots]$) pointwise dominates Work's successor (Apprentice, profile $[2, 3, 5, \ldots]$). The sacrifice—Train's immediate reward of 0 versus Work's 2—does not appear in the successor profiles, because the profiles record reachable capability, not incurred cost.

CoindHomo fails at every level: Work's immediate reward always exceeds Train's, so the action-value streams are pointwise incomparable.

Full proof: `src/CSHRL/Tasks/Verified/SkillInvestment.agda`

6.3 Preparation Dilemma: Cyclic Dynamics

A structurally different pattern with regression and context-dependent optimal actions:

At Idle, the optimal action is Prepare (sacrifice now for readiness). At Ready, the optimal action flips to Rush (capitalize on preparation). Over-preparing at Ready sends the agent *back* to Idle—a regression penalty.

Value streams: Producing = Ready = $[3, 3, 3, \ldots]$, Idle = $[1, 3, 3, \ldots]$.

CoinductiveHomomorphism handles the context-dependent ranking: Prepare > Rush at Idle (successor quality improves), Rush > Prepare at Ready (successor quality improves).

CoindHomo cannot rank the actions at Idle in *either* direction:

- Forward (Rush ≤ Prepare): head requires $1 \leq 0$—absurd.
- Reverse (Prepare ≤ Rush): head is $0 \leq 1$ (ok), but the tail requires $value(\text{Ready}) \leq_s value(\text{Idle})$, which fails at the head: $3 \leq 1$—absurd.

Full proof: `src/CSHRL/Tasks/Verified/PreparationDilemma.agda`

7 Subsumption and Classical RL

The value streams compared by CoinductiveHomomorphism are computed from the actual environment dynamics by the `solve` function—they are ground truth, not estimates. If the environment terminates (absorbing states with zero reward), the value streams reflect that. If the environment is infinite, the streams extend accordingly. The horizon is not a parameter of the condition; it is encoded in the dynamics.

7.1 CoindHomo Subsumes Discounted Return

The action-value condition compares the *full* action-value streams, which include the immediate reward as the head element. Pointwise dominance at every position implies that the cumulative sum—at every finite horizon—is at least as large for the better-ranked action. Since the discounted return is a weighted sum with positive weights, action-value dominance implies discounted return dominance for every discount factor $\gamma \in [0, 1)$.

This is genuine subsumption: any policy optimal under CoindHomo is also optimal under every discounted return criterion.

7.2 CoinductiveHomomorphism Subsumes Successor Return

The coinductive homomorphism compares successor value streams—the accumulated reward *from the state the action leads to*. The verified theorem states:

```
subsumes-successor-return : (ch : CoinductiveHomomorphism) →
  ∀ s a b N →
  CoinductiveHomomorphism._≤ₐ_ ch s a b →
  partial-sum N (successor-value s a) ≤ᵣ
  partial-sum N (successor-value s b)
subsumes-successor-return ch s a b N p =
  partial-sum-mono N _ _ (≤ₛ-to-pointwise
    (CoinductiveHomomorphism.preserves ch a b s p))
```

The top-ranked action leads to the state with the highest accumulated reward at every finite horizon. This is successor return dominance: the better-ranked action places the agent in a strictly better position.

7.3 Where Discounted Return Fails

To make the contrast with classical RL concrete, we examine Q-learning—the workhorse algorithm of model-free reinforcement learning [1]. Q-learning maintains a table of *Q-values*, $Q(s, a)$, estimating the expected discounted return of taking action a in state s and acting optimally thereafter. These estimates are updated via the Bellman equation:

$$Q(s, a) \leftarrow r(s, a) + \gamma \cdot \max_{a'} Q(s', a')$$

where $r(s, a)$ is the immediate reward, s' is the successor state, and $\gamma \in [0, 1)$ is the discount factor. The agent then selects the action with the highest Q-value: $\pi(s) = \arg\max_a Q(s, a)$. Under standard conditions, Q-learning converges to the optimal Q-values, and the greedy policy is optimal *for the discounted objective*. The problem, as we now show, is that the discounted objective itself can be wrong.

In sacrifice environments, Q-learning can converge to a suboptimal policy—not because the algorithm fails, but because the discount factor distorts the objective it optimizes. We demonstrate this concretely on the binary sacrifice MDP.

The verified ground truth. The `solve` function computes the optimal reward at each depth from the actual environment dynamics:

$$\begin{aligned} \text{solve}(\text{Paradise}, n) &= 1 \quad \text{for all } n \\ \text{solve}(\text{Trap}, n) &= 0 \quad \text{for all } n \end{aligned}$$

These are machine-verified facts, not estimates. Paradise yields reward 1 at every future depth; Trap yields 0. The accumulated reward from Paradise at horizon N is N; from Trap it is 0.

The Q-value computation. The Bellman equation gives Q-learning's converged values at state Start:

$$Q(\text{Start}, \text{GoTrap}) = \underbrace{1}_{\text{immediate}} + \gamma \cdot \underbrace{0}_{V(\text{Trap})} = 1$$

$$Q(\text{Start}, \text{GoParadise}) = \underbrace{0}_{\text{immediate}} + \gamma \cdot \underbrace{\frac{1}{1-\gamma}}_{V(\text{Paradise})} = \frac{\gamma}{1-\gamma}$$

For $\gamma < \frac{1}{2}$, we have $\frac{\gamma}{1-\gamma} < 1$, so Q-learning selects GoTrap—the action whose successor state has accumulated reward 0 at every horizon—over GoParadise, whose successor state has accumulated reward N at horizon N.

The discount factor γ geometrically suppresses $V(\text{Paradise})$, which is the only term that carries the evidence of Paradise's superiority. When γ is small enough, the suppression is sufficient to make a state worth $[1, 1, 1, \ldots]$ appear less valuable than a state worth $[0, 0, 0, \ldots]$.

The verified conclusion. We prove in Agda (`src/CSHRL/Analysis/QLearningFailure.agda`) the conjunction of two facts:

1. **Tactical:** $r(\text{Start}, \text{GoParadise}) \leq r(\text{Start}, \text{GoTrap})$ — GoTrap has the higher immediate reward.
2. **Strategic:** For every horizon N, the accumulated reward from GoParadise's successor exceeds GoTrap's:

$$\sum_{i=0}^{N-1} \text{solve}(\text{Trap}, i) \;\leq\; \sum_{i=0}^{N-1} \text{solve}(\text{Paradise}, i)$$

Together: Q-learning with $\gamma < \frac{1}{2}$ selects the action with the higher immediate reward and the strictly inferior successor state. The discount factor is the mechanism by which the tactical advantage overrides the strategic one.

Deeper sacrifice chains. The Skill Investment chain sharpens the point. Q-learning requires $\gamma > (1/5)^{1/3} \approx 0.585$ to identify the optimal training path. Below this threshold, it converges to the policy of working at Novice forever—earning 1 per step while a three-step investment would yield 5 per step permanently. CoinductiveHomomorphism identifies the training path regardless: the successor state quality is strictly better at every depth, and the environment dynamics confirm it.

7.4 CoindHomo as a Conservative Strengthening

When the two conditions coincide—in environments where immediate rewards align with the ranking—CoinductiveHomomorphism inherits CoindHomo's full subsumption of classical RL. This covers the vast majority of standard environments (Section 5).

In sacrifice environments, CoindHomo cannot rank the actions at all (the action-value streams are pointwise incomparable), while CoinductiveHomomorphism correctly identifies the better successor state. CoindHomo is the conservative choice: it refuses to rank when the immediate reward conflicts with the future. CoinductiveHomomorphism resolves the conflict in favor of successor state quality—which the environment dynamics validate.

8 The Stream Isomorphism

We defined stream dominance coinductively: $x \leq_s y$ iff heads compare and tails recursively compare. A natural question is whether this definition captures exactly pointwise dominance at every timestep. The answer is affirmative: the coinductive definition is *isomorphic* to pointwise dominance.

The forward direction (`≤ₛ-to-pointwise`) was already given in §4. The reverse:

```
pointwise-to-≤ₛ : ∀ {x y} → PointwiseDominance x y → x ≤ₛ y
head≤ (pointwise-to-≤ₛ pw) = pw zero
tail≤ (pointwise-to-≤ₛ pw) = pointwise-to-≤ₛ (pointwise-tail pw)
```

Together with `≤ₛ-to-pointwise`, this yields:

$$x \leq_s y \iff \forall n.\, x_n \leq_r y_n$$

This isomorphism has three consequences. First, it confirms that the coinductive formulation exactly captures "dominance at every timestep"—it is a precise characterization, not an approximation. Second, it establishes Finder completeness: since the Finder compares traces at finite depths, the isomorphism guarantees that trace dominance at all depths implies coinductive stream dominance. Third, the bidirectional correspondence means that rankings preserving stream order and stream orders inducing rankings are interchangeable views—the "symmetric" aspect of the framework.

The isomorphism applies equally to both CoinductiveHomomorphism and CoindHomo: both use $\leq_s$ as their codomain ordering, differing only in which streams are compared (successor value streams vs. action-value streams).

The full proof is in `appendix/StreamIsomorphism.agda`.

9 The Finder Algorithm

The preceding sections established what an optimal ranking *is*. We now turn to how to *compute* one. The Finder algorithm uses lexicographic trace comparison and requires no discount factor.

```
module Finder
  (State Action Reward : Set)
  (step           : State → Action → State × Reward)
  (_≤?_           : Reward → Reward → Bool)
  (default-action : Action)
  (all-actions    : List Action)
  where

  open import Data.List using (List; map)

  Trace : Set
  Trace = List Reward
```

9.1 Lexicographic Comparison

Compare traces element-by-element—the first difference decides:

```
_≤ₜ_ : Trace → Trace → Bool
[]       ≤ₜ []       = true
[]       ≤ₜ (_ ∷ _)  = true
(_ ∷ _)  ≤ₜ []       = false
(r₁ ∷ t₁) ≤ₜ (r₂ ∷ t₂) =
  if r₁ ≤? r₂ then
    if r₂ ≤? r₁ then (t₁ ≤ₜ t₂)
```

```
      else true
  else false
```

Unlike summation (which discards timing information), lexicographic comparison preserves it: a trace $[10, -5]$ dominates $[0, 5]$ because $10 > 0$ at step 0, regardless of subsequent elements.

9.2 Trace Evaluation

Given the comparator, we need the traces themselves. For a deterministic MDP, the trace of action a at state s to depth k is the finite prefix of the reward stream: take action a, record the reward, then act optimally (according to all actions, greedily at each step) for the remaining $k-1$ steps. `best-trace` computes this optimal continuation; `trace-action` prepends the immediate reward of a specific action:

```
mutual
  best-trace : State → ℕ → Trace
  best-trace s zero = []
  best-trace s (suc k) =
    let outcomes = map (λ a → trace-action s a k) all-actions
    in max-trace outcomes

  trace-action : State → Action → ℕ → Trace
  trace-action s a k =
    let (s' , r) = step s a
        future = best-trace s' k
    in r ∷ future

  max-trace : List Trace → Trace
  max-trace [] = []
  max-trace (t ∷ ts) = max-helper t ts

  max-helper : Trace → List Trace → Trace
  max-helper current [] = current
  max-helper current (t ∷ ts) =
    if current ≤ₜ t
    then max-helper t ts
    else max-helper current ts
```

9.3 The Ranking Finder

With traces and a comparator in hand, producing a ranking is straightforward: compute each action's trace at the desired depth, then sort by lexicographic dominance. The head of the sorted list is the optimal action; the full list is the total ranking used by the adaptation layer (§17).

```
insert : (Action × Trace) → List (Action × Trace) → List (Action × Trace)
insert x [] = x ∷ []
insert (a₁ , t₁) ((a₂ , t₂) ∷ xs) =
  if t₂ ≤ₜ t₁
  then (a₁ , t₁) ∷ (a₂ , t₂) ∷ xs
  else (a₂ , t₂) ∷ insert (a₁ , t₁) xs

sort-scored : List (Action × Trace) → List (Action × Trace)
sort-scored [] = []
sort-scored (x ∷ xs) = insert x (sort-scored xs)

find-ranking : State → ℕ → List Action
find-ranking s k =
```

```
    let scored = map (λ a → (a , trace-action s a k)) all-actions
        sorted = sort-scored scored
    in map proj₁ sorted

find-policy : State → ℕ → Action
find-policy s k =
  let ranking = find-ranking s k
  in head-default ranking
  where
    head-default : List Action → Action
    head-default [] = default-action
    head-default (x ∷ _) = x
```

No discount factor is required. Standard RL needs $\gamma < 1$ to ensure convergence of the sum; here we compare structure directly, and the lexicographic order is well-defined at any depth.

9.4 Serving Both Conditions

The Finder as defined above computes `trace-action`—whose head is the immediate reward and whose tail is the optimal future. This directly serves **CoindHomo**: action-value traces are the finite approximations of action-value streams.

For **CoinductiveHomomorphism**, the relevant comparison is between *successor-value traces*—the optimal future from the state reached by each action, *excluding* the immediate reward. The adaptation is a one-line change: instead of `r :: future`, use `future` alone. Equivalently, compare `tail(trace-action s a k)` for each action.

In aligned environments (Section 5), both comparisons yield the same ranking. In sacrifice environments, the action-value trace may be incomparable (the head reversal prevents lexicographic dominance), while the successor-value trace correctly identifies the better successor state. The Stream Isomorphism (§8) guarantees that trace dominance at all depths implies coinductive stream dominance—for whichever stream variant is used.

10 Environment Classes: Modularity Without Postulates

The Finder produces rankings; the Stream Isomorphism connects them to coinductive stream dominance. But each task still needs its own bridge lemma—a potentially tedious and error-prone process. The **Environment Class** (EC) abstraction eliminates this boilerplate. Rather than proving preservation from scratch for each task, we factor out common structure into reusable modules.

10.1 The Problem: Boilerplate and Trust

Without environment classes, every task would need:

1. Its own Finder algorithm implementation
2. Its own trace-to-stream bridge lemma
3. Its own preservation proof skeleton

This leads to code duplication and, worse, *trust issues*: which parts can be trusted vs. which need re-verification?

10.2 The Solution: Parameterized Modules

Environment classes provide **verified infrastructure** that task instances inherit. A task author fills in a parameter record (types, step function, reward order), and the class automatically provides the Finder, trace types, stream bridges, and proof scaffolding.

The full implementation is in `src/CSHRL/EnvironmentClass/`; here we show the parameter record that a task must instantiate:

```
record ECParameters : Set₁ where
  field
    State Action Reward : Set
    step     : State → Action → State × Reward
    _≤ᵣ_     : Reward → Reward → Set
    _≤?_     : (r s : Reward) → Dec (r ≤ᵣ s)
    ≤ᵣ-refl  : ∀ {r} → r ≤ᵣ r
    max      : Reward → Reward → Reward
    bottom   : Reward
    all-actions : List Action
    default-action : Action
    horizon : ℕ
```

The fields divide into three groups. The *domain* group (`State`, `Action`, `step`) specifies the MDP. The *algebraic* group (`_≤ᵣ_`, `_≤?_`, `max`, `bottom`) provides the ordered semiring structure on rewards, ensuring comparability and bounded extrema. The *enumeration* group (`all-actions`, `default-action`, `horizon`) supplies finiteness witnesses needed by the Finder's brute-force search.

Given these parameters, the environment class automatically provides:

```
-- AUTOMATICALLY PROVIDED by the EC:
-- • Trace type and orderings (_≤ₜ_, _≤ₜᵇ_)
-- • Finder algorithm (best-trace, find-ranking, find-policy)
-- • Ranking relation (_ranks_≤_)
-- • Trace-to-stream bridge infrastructure (trace-≤ₜ-head)
-- • Stream reflexivity (≤ₛ-refl)
-- • Full CoindHomo (for specialised ECs like CombinatorialPlacementMDP)
```

10.3 What Task Instances Must Provide

The burden on a task instance depends on which environment class it instantiates.

FiniteDeterministicMDP (general case). A task provides:

1. **Domain specifics:** The `State`, `Action`, `step` function.
2. **A direct preservation proof:** A proof that the Finder's ranking preserves action-value ordering—the full bridge from finite traces to infinite streams.

CombinatorialPlacementMDP (placement problems). A specialised EC for placement problems (N-Queens, Sudoku, Graph Coloring), extending FiniteDeterministicMDP with domain-specific structure. The full implementation is in `src/CSHRL/EnvironmentClass/CombinatorialPlacementMDP.agda`:

```
record PlacementParameters : Set₁ where
  field
    Config : Set
    Action : Set
    is-dead : Config → Bool
    is-solved : Config → Bool
    place : Config → Action → Config
    solved-reward : ℕ
    all-actions : List Action
    default-action : Action
    horizon : ℕ
```

```
data PlacementState (Config : Set) : Set where
  Ongoing : Config → PlacementState Config
  Dead    : PlacementState Config
  Solved : Config → PlacementState Config
```

The three constructors model the life cycle of a placement problem: `Ongoing` wraps an active configuration where pieces can still be placed; `Dead` is an absorbing failure state (e.g., two queens attacking each other with no remaining fix); `Solved` wraps a completed valid configuration. The key structural property is that Dead states produce stream $[0, 0, 0, ...]$ and Solved states produce $[R, R, R, ...]$. Any path reaching Solved therefore dominates any path reaching Dead at every timestep—precisely the condition required by the coinductive order.

The EC factors the preservation proof into reusable layers:

1. `WithAbsorbingLemmas` (parameters: `solve-Dead-0`, `solve-Solved-R`). Derives stream-level domination lemmas for absorbing states.
2. `WithBinaryStructure` (adds: `0<R`). Proves that `solve` is *binary*—every value is either 0 or R—and derives monotonicity.
3. `WithTraceBridge` (adds: `solve-horizon-suf`). Combines binary structure with horizon sufficiency to construct the full `CoindHomo`: the Finder's trace-based ranking preserves the coinductive ordering of action-value streams.

The task author supplies at most four simple ingredients (the placement parameters, two absorbing-state lemmas, and horizon sufficiency), while the EC machinery handles the entire bridge from finite traces to infinite streams.

Validation: 8-Queens ($N = 8$). The `Queens8` module (`src/CSHRL/Tasks/Verified/Queens8.agda`) instantiates the EC for the 8-Queens problem with `--safe` and zero postulates. With `solve-horizon-suf` discharged, opening `WithTraceBridge` yields a complete `CoindHomo`: for every state s and actions a, b, if the Finder ranks $a \leq b$, then the infinite reward stream of a is dominated by that of b at every time step. This is achieved for a combinatorial search space ($8^8 = 16{,}777{,}216$ candidate placements) where direct stream enumeration is infeasible.

10.4 Why This Matters: Postulate-Free Verification

- **Verified tasks** (like TwoState, Queens1, and Queens8) have **no postulates**—every claim is machine-checked.
- **Classic tasks** (like Maze, KeyDoor) can use postulates for properties that are difficult to formalise, but the *structural* preservation proof is still verified.
- **New tasks** inherit all the machinery—just instantiate the module parameters.

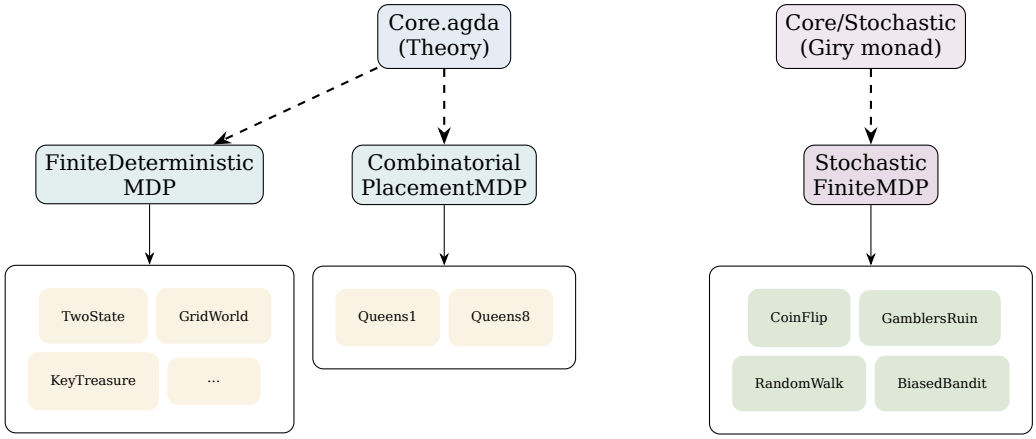


Figure 3: Environment class hierarchy. Core provides theory; ECs provide reusable verification infrastructure; task groups contain domain-specific instantiations.

11 Stochastic Extension via the Giry Monad

Everything up to this point assumes deterministic environments: each action from a given state leads to exactly one successor state and one reward. But many real-world problems are inherently stochastic—a robot's actuators have noise, a card game involves shuffled decks, a medical treatment has variable patient responses. In such environments, the step function returns not a single outcome but a *probability distribution* over outcomes.

The classical approach to stochastic MDPs uses the **Giry monad** [7]: a categorical construction that equips probability measures with a monadic interface (unit, bind), enabling compositional reasoning about probabilistic programs. In full generality, the Giry monad operates over measurable spaces and σ-algebras—far too heavy for constructive, decidable verification in Agda.

Our approach is to use a **finite distribution monad**: a discrete, constructive approximation of the Giry monad that represents distributions as weighted lists. This gives us exact arithmetic (no floating-point rounding), decidable equality, and compatibility with Agda's termination checker—while being expressive enough to model any finite-state stochastic MDP.

11.1 The Distribution Monad

A distribution over type A is represented as a list of outcome–weight pairs, where weights are natural numbers encoding unnormalized probability mass. For example, a fair coin flip producing `Won` or `Lost` is `(Won, 1) :: (Lost, 1) :: []`—each outcome has weight 1 out of total weight 2. This "poor man's measure theory" suffices for finite MDPs and supports exact arithmetic with no rounding.

```
Dist : Set → Set
Dist A = List (A × ℕ)

pure : ∀ {A} → A → Dist A
pure a = (a , 1) :: []

scale : ∀ {A} → ℕ → Dist A → Dist A
scale k = map (λ { (a , w) → a , k * w })

_»=_ : ∀ {A B} → Dist A → (A → Dist B) → Dist B
d »= f = concatMap (λ { (a , w) → scale w (f a) }) d
```

The three operations form a monad: `pure` creates a point mass; `scale` multiplies all weights (for conditional branching); and `>>=` implements the law of total probability—each outcome in `d` contributes its weight-scaled continuation. The monad laws are verified in `src/CSHRL/Probability/Finite.agda`.

11.2 Stochastic Step Functions

The key change: the step function returns a distribution over outcomes.
Example (CoinFlip):

```
data CoinState : Set where
  Ready Won Lost : CoinState

data CoinAction : Set where
  Flip Stay : CoinAction

bernoulli : ∀ {A} → A → ℕ → A → ℕ → Dist A
bernoulli a₁ w₁ a₂ w₂ = (a₁ , w₁) ∷ (a₂ , w₂) ∷ []

coin-step : CoinState → CoinAction → Dist (CoinState × ℕ)
coin-step Ready Flip = bernoulli (Won , 1) 1 (Lost , 0) 1
coin-step Ready Stay = pure (Ready , 0)
coin-step Won _ = pure (Won , 0)
coin-step Lost _ = pure (Lost , 0)
```

11.3 Lexicographic Coinductive Comparison

In deterministic MDPs, the value stream from a state is a single fixed sequence—pointwise comparison works perfectly. In stochastic MDPs, however, the "value" at each depth is an *expected* reward, computed by marginalizing over all possible trajectories. The problem is that two actions can have identical expected rewards at depth 0 but different distributions. Pointwise comparison $\leq_s$ would demand dominance at *every* depth, which is too strict: if action a already wins at depth 0, requiring it to also win at depth $1, 2, \ldots$ rules out many valid rankings.

The correct notion is **lexicographic coinductive comparison**: compare expected heads; if they differ, the comparison is decided. Only recurse to tails when heads are equal:

```
record _≤ₛ-lex_ {Reward : Set} (_≤ᵣ_ : Reward → Reward → Set)
                (x y : Stream Reward) : Set where
  coinductive
  field
    head≤ : head x ≤ᵣ head y
    tail≤ : head x ≡ head y → _≤ₛ-lex_ _≤ᵣ_ (tail x) (tail y)
```

This is still coinductive—the infinite tower structure is preserved—but captures the correct semantics: earlier rewards break ties, and only equal heads require recursive comparison.

11.4 Why Lexicographic Works

Consider the CoinFlip example at state `Ready`:

- `Stay`: expected head = 0
- `Flip`: expected head = 1 (unnormalized)

For the ranking `Stay` ≤ `Flip`:

- `head≤`: $0 \leq 1$ □
- `tail≤`: requires $0 \equiv 1$, which is **impossible**

Since the condition $0 \equiv 1$ has no inhabitants, the tail proof is trivially satisfied via absurd pattern matching—no recursive argument needed. This makes CoinFlip **fully verified with no postulates**.

11.5 The Proof Pattern

All four verified stochastic examples (CoinFlip, GamblersRuin, RandomWalk, BiasedBandit) follow the same pattern:

1. One action dominates in expected immediate reward
2. Head preservation is trivial from the ranking hypothesis
3. The tail obligation requires head(a) ≡ head(b), which is absurd
4. The empty pattern `()` completes the proof

```
coinflip-tail-example : ∀ {A : Set} → 0 ≡ 1 → A
coinflip-tail-example ()
```

This makes CoinFlip—and all four examples—**fully verified with no postulates**.

11.6 The Stochastic Coinductive Homomorphism

We can now assemble the stochastic analogue of the deterministic `CoindHomo` record from §4. The structure is identical—a ranking paired with a preservation proof—but the codomain changes: instead of comparing deterministic reward streams with pointwise $\leq_s$, we compare *expected* reward streams with lexicographic $\leq_{lex}$.

```
record StochasticCoindHomo
  (State Action : Set) (StreamR : Set)
  (expected-action-value : State → Action → Stream StreamR)
  (_≤ᵣ_ : StreamR → StreamR → Set) : Set₁ where
  field
    _≤ₐ_ : State → Action → Action → Set
    preserves : ∀ a b s → _≤ₐ_ s a b →
                  _≤ₛ-lex_ _≤ᵣ_ (expected-action-value s a) (expected-action-value s b)
```

Reading the fields: `_≤ₐ_` is the ranking, indexed by state, just as in the deterministic case. The `preserves` field says: if the ranking considers b at least as good as a at state s, then the expected action-value stream of a is lexicographically dominated by that of b. The parameter `expected-action-value` is computed by the stochastic Environment Class: it marginalizes the `Dist`-valued step function over all possible successors at each depth, producing an infinite stream of expected rewards.

The full implementation is in `src/CSHRL/Core/Stochastic.agda`, with the stochastic Environment Class in `src/CSHRL/EnvironmentClass/StochasticFiniteMDP.agda`.

11.7 Both Conditions in the Stochastic Setting

Just as the deterministic framework distinguishes CoinductiveHomomorphism (successor state quality) from CoindHomo (full action-value streams), the stochastic extension admits both:

- **StochasticCoindHomo**: preserves expected *action-value* streams lexicographically. This bundles the immediate expected reward and the expected successor quality.
- **StochasticCoinductiveHomomorphism**: preserves expected *successor-value* streams lexicographically. This decouples from immediate reward, allowing sacrifice-pattern reasoning in stochastic environments.

The decomposition theorem (Theorem 1) lifts directly: the stochastic action-value condition decomposes into expected-immediate-reward compatibility plus stochastic successor quality preservation.

Remark 1 (Distributional Hierarchy and CoinductiveHomomorphism). The FOSD and SD[k] extensions (§12–13) are *per-timestep* conditions: they require distributional dominance at each individual depth. This aligns naturally with `StochasticCoindHomo`, whose action-value stream bundles immediate reward and successor quality at every timestep.

`StochasticCoinductiveHomomorphism`, by contrast, compares successor-value streams *lexicographically*: dominance is required only at the first depth where expected values differ. Expected successor values may be equal at all finite depths, with the comparison satisfied vacuously. This holistic, stream-level guarantee does not decompose into per-timestep distributional claims, placing lexicographic CoinductiveHomomorphism outside the scope of the per-timestep FOSD/SD hierarchy.

In the deterministic case, $\leq_s$ is unconditionally pointwise (head $\leq$ AND tail $\leq_s$, no equality guard), so the Stream Isomorphism (§8) recovers per-depth dominance. The stochastic split—$\leq_{lex}$ guards tail recursion on head equality—is what creates the gap.

11.8 Relationship Between Orderings

- **Pointwise $\Rightarrow$ Lexicographic**: If $x \leq_s y$ at every timestep, then $x \leq_{lex} y$.
- **Lexicographic $\not\Rightarrow$ Pointwise**: Lexicographic is more permissive.
- **Deterministic case**: Both coincide, since trajectories are fixed and the comparison is deterministic.

Why lexicographic is correct for stochastic: Expected values collapse distributions to scalars. When $\mathbb{E}[\text{head}(a)] < \mathbb{E}[\text{head}(b)]$, action b is already established as better—demanding further tail proofs adds no information. Lexicographic comparison captures exactly this semantics.

11.9 Subsumption of Classical RL

By linearity of expectation:

$$\mathbb{E}\left[\sum_{t=0}^{N-1} r_t\right] = \sum_{t=0}^{N-1} \mathbb{E}[r_t]$$

Comparing partial sums of expected rewards is equivalent to comparing expected partial sums—exactly what classical RL optimizes. The stochastic subsumption proof is in `appendix/StochasticSubsumption.agda`.

The stochastic framework so far compares actions by their expected reward streams. But expected values are themselves a lossy summary—they collapse an entire distribution to a single number. Can we do better? The next two sections develop a hierarchy of distributional comparisons, from FOSD (the strongest) through progressively weaker SD[k] orderings, each capturing a broader class of rational agents.

12 First-Order Stochastic Dominance

The expected-value stochastic pipeline (§11) compares actions by $\mathbb{E}[\text{trace}(s,a)] \leq \mathbb{E}[\text{trace}(s,b)]$. This suffices for expected-value maximizers, but collapses the full reward distribution to a single scalar.

A strictly stronger comparison is **First-Order Stochastic Dominance (FOSD)**: action a FOSD-dominates b at state s iff for all thresholds r and all timesteps n:

$$P(\text{reward}_n(s,a) \leq r) \leq P(\text{reward}_n(s,b) \leq r)$$

i.e., a's cumulative distribution lies everywhere below b's. FOSD implies expected-value dominance ($\mu \geq_{\text{FOSD}} \nu \Rightarrow \mathbb{E}[\mu] \geq \mathbb{E}[\nu]$) but not conversely. An action FOSD-dominating another is preferred by *all monotone utility functions*—risk-averse, risk-neutral, and risk-seeking agents agree.

The complete FOSD pipeline is implemented and machine-verified across three modules (`Probability.FOSD`, `Core.FOSD`, `Synthesis.FOSDStochasticFiniteMDP`).

12.1 Foundation: Probability.FOSD

To verify FOSD in Agda, we need to compute and compare CDFs over our finite distributions. Consider two distributions: $\mu = \{0:1, 1:2\}$ (outcome 0 with weight 1, outcome 1 with weight 2) and $\nu = \{0:2, 1:1\}$. Distribution μ FOSD-dominates ν iff $\text{CDF}(\mu, r) \leq \text{CDF}(\nu, r)$ for every threshold r: at r=0, $\text{CDF}(\mu, 0) = 1 \leq 2 = \text{CDF}(\nu, 0)$ □; at r=1, both CDFs equal 3 □. So $\mu \geq_{\text{FOSD}} \nu$: the "$\mu$-agent" is more likely to achieve higher rewards.

The `Probability.FOSD` module mechanizes this reasoning:

CDF and ordering. Functions `cdf-weight` and `comp-weight` compute the CDF and complementary CDF for our list-based distributions, with the identity $\text{CDF}(d, r) + \text{comp}(d, r) = W$ verified. The FOSD ordering (`_FOSD≤_`) is defined as pointwise CDF dominance, with reflexivity and transitivity proved.

Bridges. The theorem `fosd→ev` connects FOSD to expected values: if $\mu \geq_{\text{FOSD}} \nu$ and both have equal total weight, then $\mathbb{E}[\mu] \geq \mathbb{E}[\nu]$. The CDF-over-concatenation lemmas (`cdf-weight-++`, `total-weight-++`) establish that CDF and total weight are additive—enabling compositional reasoning when distributions are combined.

Decidability. The Boolean function `fosd?` compares total weights and CDF values up to the joint maximum support. Machine-verified **soundness** (`fosd?-sound`: `fosd?` $\mu\ \nu$ = `true` $\Rightarrow$ μ $\text{FOSD}_{\leq}$ ν) and **completeness** (`fosd?-complete`: the converse) establish it as a decision procedure.

12.2 Coinductive FOSD and the Bridges

A single FOSD comparison between two distributions answers: "is one unambiguously better?" But an RL agent faces a sequence of reward distributions, one per timestep. `Core.FOSD` lifts FOSD from individual distributions to this infinite sequence.

PointwiseFOSD requires FOSD dominance at every depth: $\forall n.\ \text{marginal}(s, b, n)\ \text{FOSD}_{\leq}\ \text{marginal}(s, a, n)$. In words: at timestep 0, action a's marginal reward distribution FOSD-dominates b's; at timestep 1, again; and so on forever.

StreamFOSD (`_≤ₛ-fosd_`) is the coinductive reformulation: a record that unfolds one depth at a time, just like the deterministic $\leq_s$. The **Stochastic Isomorphism** proves these two formulations are equivalent—extending the Stream Isomorphism (§8) from scalar streams to distribution streams. (The Agda identifiers `conjecture-forward` and `conjecture-reverse` retain their historical names; both directions are now machine-verified.)

With these tools in hand, we define `FOSDCoindHomo`: a ranking that preserves `PointwiseFOSD` at every state. This is strictly stronger than `StochasticCoindHomo`—it guarantees optimality not just for expected-value agents, but for all agents with monotone utility functions:

```
record FOSDCoindHomo : ₁Set where
  field
    ≤ₐ__ : State → Action → Action → Set
    preserves-fosd : ∀ a b s →
      ≤ₐ__ s a b → PointwiseFOSD s b a
```

Two bridge theorems connect the FOSD and expected-value worlds:

- **FOSD ⇒ E[X] per-timestep** (`pointwise-ev`): applies `fosd→ev` at each depth.
- **FOSD-to-Lex Bridge** (`fosd→lex-stream`): lifts per-timestep EV dominance to lexicographic EV-stream dominance.

```
→fosdlex-stream : ∀ s a b →
  PointwiseFOSD s a b →
  marginal-ev-stream s b ≤ₛ-ev marginal-ev-stream s a

fosd-→homoev-lex : (homo : FOSDCoindHomo) →
  ∀ a b s → FOSDCoindHomo.≤ₐ__ homo s a b →
    marginal-ev-stream s a ≤ₛ-ev marginal-ev-stream s b
```

`fosd-homo→ev-lex` establishes the key implication: every `FOSDCoindHomo` is automatically a `StochasticCoindHomo`. Rankings verified via FOSD simultaneously guarantee expected-value optimality *and* dominance for all monotone utilities.

Full implementation: `src/CSHRL/Probability/FOSD.agda` and `src/CSHRL/Core/FOSD.agda`.

13 The Stochastic Dominance Hierarchy

FOSD provides the strongest distributional comparison: an action FOSD-dominating another is preferred by *all* monotone utility functions. But this strength is also a limitation—many distributions that exhibit a clear "better" relationship fail the FOSD test because their CDFs cross at some threshold.

The **Stochastic Dominance Hierarchy** generalizes FOSD to a parameterized family of orderings, each capturing a broader class of utility functions. The k-th order SD condition integrates the CDF k times; each additional integration "smooths out" CDF crossings:

$$\text{SD}[0] = \text{FOSD} \;\Rightarrow\; \text{SD}[1] = \text{SOSD} \;\Rightarrow\; \text{SD}[2] = \text{TOSD} \;\Rightarrow\; \cdots$$

13.1 Foundation: Probability.SD

The idea behind the hierarchy is elegant: instead of comparing CDFs directly (which may cross), compare their *integrals*. Integrating a CDF smooths out local crossings; integrating again smooths further. At each level, more distribution pairs become comparable—at the cost of a narrower class of agents who would unanimously prefer one over the other.

The hierarchy is built on a single operation—**prefix sum** (the discrete analogue of integration):

```
prefix-sum : ℕ( → ℕ) → ℕ → ℕ
prefix-sum f zero    = f 0
prefix-sum f (suc r) = f (suc r) + prefix-sum f r

sd-weight : ℕ → Dist ℕ → ℕ → ℕ
sd-weight zero    d = cdf-weight d
sd-weight (suc k) d = prefix-sum (sd-weight k d)
```

`sd-weight 0` is the CDF itself (FOSD level); `sd-weight 1` is the integrated CDF (SOSD level); `sd-weight` k is the k-fold prefix sum. The ordering at each level is:

```
_SD[_≤]_ : Dist ℕ → ℕ → Dist ℕ → Set
μ SD[ k ≤] ν = ∀ r → sd-weight k ν r ≤ sd-weight k μ r
```

The one-line subsumption. The key property of the hierarchy is that subsumption is a single application of prefix-sum monotonicity:

```
prefix-sum-mono : ∀ {f g} →
  ∀( r → f r ≤ g r) →
  ∀ r → prefix-sum f r ≤ prefix-sum g r

SD-subsumes : ∀ k μ{ ν} →
  μ SD[ k ≤] ν → μ SD[ suc k ≤] ν
SD-subsumes k sd = prefix-sum-mono sd
```

If the k-th order integrated CDFs are pointwise ordered, their prefix sums (the $(k{+}1)$-th order) are too. This gives the full chain FOSD ⇒ SOSD ⇒ TOSD ⇒ ⋯ in one line.

The SD-to-EV bridge. A key lemma shows that `sd-weight` k d `0` ≡ `cdf-weight` d `0` for all k—prefix-sum at $r{=}0$ is just the function value. This means SD[k] at $r{=}0$ gives CDF dominance at $r{=}0$, which yields expected-value dominance via the existing FOSD→EV bridge:

```
sd-at-0 : ∀ k d → sd-weight k d 0 ≡ cdf-weight d 0

→sdev : ∀ k μ ν →
```

```
  AllBelow μ 1 → AllBelow ν 1 →
  total-weight μ ≡ total-weight ν →
  μ SD[ k ≤] ν →
  weighted-sum μ ≤ weighted-sum ν
```

Strict separation. SOSD is *strictly* weaker than FOSD. Consider $\mu = \{0:2, 2:1\}$ and $\nu = \{1:3\}$. At r=1: $\text{CDF}(\nu, 1) = 3 > 2 = \text{CDF}(\mu, 1)$, so FOSD fails. But the integrated CDFs satisfy $\text{icdf}(\nu, r) \leq \text{icdf}(\mu, r)$ at every r—confirmed computationally by `refl`.

13.2 Coinductive Lift: Core.SD

Just as FOSD was lifted from single distributions to infinite streams (§12), the entire SD hierarchy lifts to reward streams. The pattern is the same: require the distributional ordering at every timestep, then package this into a coinductive homomorphism record parameterized by the SD level k.

`Core.SD` provides:

```
PointwiseSD : ℕ → State → Action → Action → Set
PointwiseSD k s a b =
  ∀ n → marginal-reward s b n SD[ k ≤] marginal-reward s a n

record SDCoindHomo (k : ℕ) : ₁Set where
  field
    ≤ₐ__ : State → Action → Action → Set
    preserves-sd : ∀ a b s →
      ≤ₐ__ s a b → PointwiseSD k s b a
```

`SDCoindHomo 0` is `FOSDCoindHomo`; `SDCoindHomo 1` is a ranking optimal for all risk-averse agents; `SDCoindHomo` k captures the (k+1)-th order utility class.

Hierarchy subsumption on homos. If a ranking preserves SD[k] (the strongest condition), it automatically preserves SD[k+1] (the weaker condition):

```
sd-homo-subsumes : ∀ k → SDCoindHomo k → SDCoindHomo (suc k)
```

A single FOSD verification (k=0) simultaneously establishes optimality for *all* levels of the hierarchy.

The SD Isomorphism. The coinductive/pointwise equivalence established for FOSD (§12) generalizes to the full hierarchy. We define a coinductive `StreamSD` k—a record with `head-sd` (SD[k] at the current depth) and `tail-sd` (coinductively for subsequent depths)—and prove the isomorphism:

```
sd-iso-forward : ∀ k s a b →
  StreamSD k (marginal-stream s a) (marginal-stream s b)
  → PointwiseSD k s a b

sd-iso-reverse : ∀ k s a b →
  PointwiseSD k s a b
  → StreamSD k (marginal-stream s a) (marginal-stream s b)

sd-isomorphism : ∀ k → SD-Isomorphism k
```

The proof is purely structural—the same head/tail decomposition that works for FOSD (k=0) works uniformly for all k, since the argument never inspects the relation on distributions. This unifies the distributional and coinductive views at every level of the hierarchy: the Stochastic Isomorphism from §12 is the special case k=0. Full implementation in `src/CSHRL/Core/SD.agda`.

The master bridge. The central theorem: any `SDCoindHomo` k (at any level) yields lexicographic EV-stream dominance:

```
sd-→homoev-lex : (homo : SDCoindHomo k) →
  ∀ a b s → SDCoindHomo.≤ₐ__ homo s a b →
    marginal-ev-stream s a ≤ₛ-ev marginal-ev-stream s b
```

The proof uses `sd→ev` at each timestep, then lifts to the stream level via `pw→lex`. This closes the hierarchy: rankings verified at *any* SD level yield expected-value optimality, while higher levels provide progressively weaker (easier to satisfy) requirements for broader classes of utility functions.

Remark 2 (Utility Function Classes). The SD hierarchy parameterizes the tradeoff between verification strength and agent generality:

- **SD**[0] **(FOSD):** optimal for all monotone utilities (risk-seeking, neutral, and averse).
- **SD**[1] **(SOSD):** optimal for all concave monotone utilities (risk-averse agents).
- **SD**[2] **(TOSD):** optimal for all utilities with non-increasing absolute risk aversion (prudent agents).
- **SD**[k]**:** each level captures a broader class, with weaker verification requirements.

When FOSD cannot be established between two distributions (CDF crossing), SOSD may still hold—accepting distributions with the same "integrated tail behavior." The hierarchy lets the framework verify the strongest relationship that exists, rather than giving up when FOSD fails.

Full implementation: `src/CSHRL/Probability/SD.agda` and `src/CSHRL/Core/SD.agda`.

14 Compositional Ranking Algebra

As environments grow in complexity, monolithic verification becomes impractical. Consider a warehouse robot that must navigate (an MDP with 20 states) *and* pick items (an MDP with 50 configurations). Verifying the product MDP ($20 \times 50 = 1000$ states, $O(|A|^2)$ pairs per state) is far more expensive than verifying each component independently. The Compositional Ranking Algebra proves that verified rankings *compose*—enabling a "divide and conquer" strategy: verify navigation and picking separately, then combine.

The mathematical foundation is that `prefix-sum` is a *linear operator*: it distributes over addition and scalar multiplication. Since the distributional ordering SD[k] is built from iterated prefix sums, this linearity yields closure properties:

```
SD-++ : ∀ k μ₁{ ν₁ μ₂ ν₂ : Dist ℕ} →
  μ₁ SD[ k ≤] ν₁ → μ₂ SD[ k ≤] ν₂ →
  μ₁( ++ μ₂) SD[ k ≤] ν₁( ++ ν₂)

SD-scale : ∀ k c μ{ ν : Dist ℕ} →
  μ SD[ k ≤] ν → scale c μ SD[ k ≤] scale c ν
```

Both proofs are one-liners after rewriting. Distribution concatenation preserves SD[k] because addition is monotone; scaling preserves it because multiplication by a non-negative constant is monotone.

14.1 Verified Rankings and Composition

The algebra centers on a single data structure: a `VerifiedRanking` bundles a ranking with a machine-checked proof that it preserves SD[k] at every state and timestep. Once two components each carry a `VerifiedRanking`, the algebra provides operations to produce a `VerifiedRanking` for their combination—with no additional proof burden on the user.

```
record VerifiedRanking
  (State Action : Set)
  (marginal : State → Action → ℕ → Dist ℕ)
  (k : ℕ) : ₁Set where
  field
    ≤ₐ__      : State → Action → Action → Set
    preserves : ∀ a b s → ≤ₐ__ s a b →
                ∀ n → marginal s a n SD[ k ≤] marginal s b n
```

The algebra provides five operations, all machine-verified. To illustrate: suppose we have verified that "navigate left" dominates "navigate right" in a navigation sub-MDP ($\texttt{vr}_1$), and that "pick item A" dominates "pick item B" in a picking sub-MDP ($\texttt{vr}_2$). `++-product` produces a verified ranking for the product MDP (navigate *and* pick), where "left + A" dominates "right + B". The proof obligation is zero—the algebra handles it internally.

- **Hierarchy subsumption:** Any ranking verified at level k is automatically verified at the weaker level k+1.
- **Product composition** (`++-product`): Two independently-verified components yield a verified product via distribution concatenation.
- **Convolution product** (`conv-product`): For product MDPs with additive rewards, the independent sum of FOSD-dominated distributions is FOSD-dominated—proved via discrete Abel summation.
- **Sum composition:** Disjoint environments compose with dispatched rankings.
- **Scaling:** Reward amplification preserves verification.

These operations compose freely: for instance, `scaled-product c vr₁ vr₂ = scale-ranking c (++-product vr₁ vr₂)` scales a product ranking in one expression. This algebraic structure enables modular, "divide and conquer" verification of complex multi-component MDPs.

Full implementation: `src/CSHRL/Probability/Compose.agda` and `src/CSHRL/Core/Compose.agda`.

15 Verified State Abstraction

Everything so far applies to finite MDPs where the state space can be enumerated. But many real-world problems have infinite or very large state spaces—a robot's continuous position, a game with billions of configurations. Verified state abstraction bridges this gap: verify a ranking on a small, finite *abstract* system, then lift it to the full concrete system.

A `StateAbstraction` bundles three components:

- **project**: Concrete → Abstract—collapses concrete states to abstract classes.
- **embed**: Abstract → Concrete—selects a representative for each class.
- **section**: `project` ∘ `embed` = id—the representatives are consistent.

No finiteness assumption is placed on `Concrete`: it can be $\mathbb{N}$, $\mathbb{R}^n$, or any type in `Set`.

15.1 The Lifting Theorem

The central result is `abstract-lift`:

> *Given a state abstraction and a proof that the marginal reward function is constant within each abstract class (marginal-invariance), any `VerifiedRanking` on the abstract system lifts to a `VerifiedRanking` on the full concrete system.*

This is the standard model-irrelevance condition from the state abstraction literature [13], but formalized constructively in Agda with a machine-checked proof. The verification cost is $O(|\text{Abstract}|)$ regardless of the concrete space's cardinality.

15.2 Composition

Three composition operators extend the framework:

- `product-abstraction`: component-wise abstraction of product environments.
- `compose-abstraction`: two-level abstractions chain ($C \xrightarrow{\alpha_1} M \xrightarrow{\alpha_2} A$).
- `abstract-conv-product`: the full pipeline—abstract each component, verify FOSD on abstract systems, compose via convolution, and lift to the concrete product.

Validated on infinite-state environments: Regional Policy ($\mathbb{N}$ states, `Bool` abstraction), Three-Zone ($\mathbb{N}$ states, 3-class abstraction), and CartPole ($\mathbb{N}^4$ discretized states, angle-based abstraction)—all postulate-free.

Full implementation: `src/CSHRL/Core/Abstraction.agda`.

16 Learning as Symmetry Restoration

The Finder computes rankings at a fixed depth. But real learning is *incremental*: we observe samples, detect violations, and restore them.

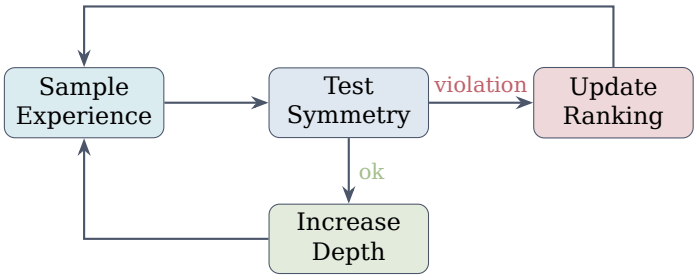


Figure 4: The learning loop: sample, test for violations, update ranking, refine depth.

16.1 Violations and Swaps

Suppose the current ranking at some state is [Stay, Go] (Stay preferred), but the Finder discovers at depth k that Go's trace dominates Stay's. This is a **violation**: the ranking disagrees with the evidence. The fix is to swap Go ahead of Stay, producing [Go, Stay]. Each swap fixes exactly one violated pair without disturbing unrelated pairs.

Formally, a violation records which state, which two actions, and at what depth the disagreement was found:

```
record Violation : Set where
  constructor violation
  field
    viol-state  : State
    viol-better : Action
    viol-worse  : Action
    viol-depth  : ℕ

DominanceOracle : Set
DominanceOracle = State → Action → Action → Bool
```

The `DominanceOracle` is the ground-truth pairwise comparator—in practice, instantiated by the Finder's trace comparison at the current depth. The fix is to **swap** the violated pair in the ranking:

```
swap-in-list : Action → Action → List Action → List Action
swap-in-list _ _ [] = []
swap-in-list better worse (x :: xs) with x ≟ₐ worse
... | yes _ = better :: worse :: remove-action better xs
... | no _ = x :: swap-in-list better worse xs

global-swap-updater : Violation → ExplicitRanking → ExplicitRanking
global-swap-updater v ranking s =
  swap-in-list (Violation.viol-better v) (Violation.viol-worse v) (ranking s)
```

16.2 Monotonicity Guarantee

The critical question is: does repeated swapping converge? The answer is yes, and we can bound the number of steps. The argument has three parts:

Theorem 2 (Swap Monotonicity)**.** *Let v be a violation with actions $(better, worse)$. After swapping: (1) the violated pair is correctly ordered; (2) unrelated pairs are unchanged; (3) total violations decrease by at least 1.*

In Agda, part (1) is `SwapFixesPair`: after swapping, the oracle confirms the pair is correctly ordered. Part (2) is `SwapPreservesUnrelated`: for any pair (a, b) where neither a nor b is involved in the swap, their relative order is unchanged. Together, these guarantee that each swap reduces the violation count by exactly one without introducing new violations.

```
SwapFixesPair : Set
SwapFixesPair = ∀ (better worse : Action) (xs : List Action) →
  (better ≡ worse → ⊥) →
  is-dominated-by (swap-in-list better worse xs) worse better ≡ true

SwapPreservesUnrelated : Set
SwapPreservesUnrelated = ∀ (better worse a b : Action) (xs : List Action) →
  (a ≡ better → ⊥) → (a ≡ worse → ⊥) →
  (b ≡ better → ⊥) → (b ≡ worse → ⊥) →
  is-dominated-by xs a b ≡ is-dominated-by (swap-in-list better worse xs) a b
```

Since each swap strictly decreases the violation count and violations are non-negative, the process terminates. The worst case is when every pair at every state is initially violated, giving a convergence bound:

$$\text{max-violations} \leq |S| \times \binom{|A|}{2} = |S| \times \frac{|A|(|A|-1)}{2}$$

The `ConvergenceTheorem` packages the full guarantee: given a valid violation finder and a ground-truth oracle, iterated swapping from any initial ranking reaches zero violations in at most `max-violations` steps. Reading the type: the hypotheses require (i) `StrictDecrease`—each swap reduces the count; (ii) `FindViolValid`—the finder only reports genuine violations; (iii) oracle soundness—reported better actions truly dominate; and (iv) completeness—when no violation is found, the count is zero.

```
ConvergenceTheorem : DominanceOracle → List State → Set
ConvergenceTheorem oracle all-states =
  StrictDecrease oracle all-states →
  ∀ (ranking₀ : ExplicitRanking)
    (find-viol : ExplicitRanking → Maybe Violation) →
  FindViolValid find-viol →
  (∀ r v → find-viol r ≡ just v →
     oracle (Violation.viol-state v) (Violation.viol-better v)
            (Violation.viol-worse v) ≡ true) →
  (∀ r → find-viol r ≡ nothing →
```

```
    count-total-violations r oracle all-states ≡ 0) →
∃[ n ] (n ≤ count-total-violations ranking₀ oracle all-states ×
        count-total-violations (iterated-update find-viol ranking₀ n)
          oracle all-states ≡ 0)
```

Learning is guaranteed to converge in at most `max-violations` swaps. Full proofs are in `src/CSHRL/Learning/Base.agda`.

Remark 3 (Full-ranking learning dissolves the exploration/exploitation trade-off). The convergence bound $|S| \times \binom{|A|}{2}$ counts *all* pairwise violations, including those between the two worst actions. To reach zero violations the learner must correctly order every pair—not just identify the argmax. Uniform pairwise sampling suffices for convergence; no separate exploration mechanism (ϵ-greedy, entropy bonus, UCB) is needed.

16.3 Practical Learning Interface

The learning infrastructure provides a stateful interface supporting:

- **Checkpointing:** The `LearnerState` captures training progress; save to pause, resume by passing to `train-step`.
- **Incremental learning:** Process samples one at a time or in batches.
- **Active refinement:** The `ActiveLearner` maintains an explicit ranking updated on violation (swap), rather than re-querying the Finder.
- **Policy materialization:** A `PolicyTable` is a concrete list of (state, action-ranking) pairs—the verified analogue of trained weights. At deployment, `lookup` retrieves the cached ranking in $O(n)$; for states not in the table a fallback to the Finder guarantees no state is ever unhandled.

The same learning loop applies to both CoinductiveHomomorphism and CoindHomo: the only difference is whether the dominance oracle compares successor-value traces or action-value traces.

The output of this learning process is a *total ranking* at every state: every action has a definite position. This totality is exactly what enables the $O(1)$ adaptation described next—when an action becomes unavailable, we simply skip it in the ranking and pick the next one, with no recomputation needed.

16.4 Stochastic and Placement Learning

The learning infrastructure extends to both stochastic MDPs (comparing expected traces; module in `src/CSHRL/Learning/StochasticFiniteMDP.agda`) and `CombinatorialPlacementMDP` (short-circuit traces for absorbing Dead/Solved states; module in `src/CSHRL/Learning/CombinatorialPlacementMDP.agda`). All monotonicity guarantees carry over.

17 Instant Adaptation to Action Unavailability

In many practical settings—robotics, network routing, game playing—actions become unavailable at runtime. Classical RL stores only the argmax action and must recompute the policy at cost $O(|S| \cdot |A|)$ or worse.

Because the framework maintains a *total ranking* over all actions, adaptation is $O(1)$: when the top-ranked action becomes unavailable, the second-ranked action is already known.

Theorem 3 (Restricted Preservation)**.** *If ranking R satisfies the homomorphism property for actions $\mathcal{A}$, and $\mathcal{A}' \subseteq \mathcal{A}$ is the set of available actions, then the restriction $R|_{\mathcal{A}'}$ satisfies the same property for $\mathcal{A}'$.*

The preservation property is pairwise: the fact that $a \leq_a b$ implies stream dominance does not depend on which other actions exist. Removing unavailable actions leaves the relative order of remaining actions unchanged.

```
Available : Set
Available = Action → Bool

filter-available : Available → List Action → List Action
filter-available _ [] = []
filter-available p (x :: xs) =
  if p x then x :: filter-available p xs else filter-available p xs

restricted-explicit-ranking : Available → ExplicitRanking → ExplicitRanking
restricted-explicit-ranking avail ranking s = filter-available avail (ranking s)
```

Beyond filtering, actions can be **demoted** rather than removed—pushed to the end of the ranking so they are used only as a last resort:

```
filter-bool : (Action → Bool) → List Action → List Action
filter-bool _ [] = []
filter-bool p (x :: xs) =
  if p x then x :: filter-bool p xs else filter-bool p xs

demote-unavailable : Available → List Action → List Action
demote-unavailable avail xs = available ++ unavailable
  where
    available = filter-bool avail xs
    unavailable = filter-bool (λ a → not (avail a)) xs


make-action-unavailable : Action → ExplicitRanking → ExplicitRanking
make-action-unavailable forbidden ranking s = demote-to-end forbidden (ranking s)
```

Classical RL stores only the argmax; when that action fails, the policy must be recomputed at cost $O(|S| \cdot |A|)$. The framework stores a total ranking, so when the top action fails the second-best is already available—$O(1)$ lookup.

This applies to both conditions: CoinductiveHomomorphism's successor-value preservation and CoindHomo's action-value preservation are both pairwise properties that restrict without loss.

18 Scope, Assumptions, and Limits

It is important to distinguish the *coinductive ideal* from the *algorithmic machinery*:

- **Coinductive optimality (ideal):** The definitions in §4 are about infinite streams and pointwise dominance at every time step.
- **Finder (approximation):** The Finder (§9) compares *finite* traces lexicographically at a fixed depth. It becomes exact under conditions verified by the Environment Class infrastructure (e.g., horizon sufficiency for placement problems).
- **Learner (policy discovery):** The Learner (§16) wraps Finder-style comparisons into a stateful learning loop.

The deterministic framework assumes: finite action space, deterministic step function, decidable reward order, and a horizon parameter for Finder approximation. Section 11 lifts to stochastic environments; Section 15 lifts to infinite state spaces.

A natural objection: the coinductive homomorphism is undecidable in general. We view this as analogous to AIXI [4]: an uncomputable ideal that organizes a hierarchy of practical approximations. Crucially, Level ω (the full coinductive homomorphism) *is* achievable for structured domains: the 8-Queens instance attains it by exploiting binary reward structure and finite game depth.

19 Verified Instantiations: From Theory to Tasks

The examples in this section validate that the formal machinery instantiates correctly. Each one that type-checks is a proof that the framework applies to that task class.

19.1 TwoState: Minimal Working Example

```
module TwoStateExample where
  open import Data.Nat using (_≤_; z≤n; s≤s)
  open import Data.Nat.Properties using (≤-refl)

  data TwoState : Set where
    Start : TwoState
    Goal : TwoState

  data TwoAction : Set where
    Go : TwoAction
    Stay : TwoAction

  two-step : TwoState → TwoAction → TwoState × ℕ
  two-step Start Go = (Goal , 1)
  two-step Start Stay = (Start , 0)
  two-step Goal _    = (Goal , 1)

  two-actions : List TwoAction
  two-actions = Go :: Stay :: []

  open Core TwoState TwoAction ℕ two-step _≤_ _⊔_ 0 two-actions
```

At `Start`, `Go`'s future is $(1, 1, 1, ...)$ and `Stay`'s is $(0, 0, 0, ...)$; the ranking `[Go, Stay]` satisfies both CoinductiveHomomorphism and CoindHomo (the conditions coincide here since the immediate rewards are aligned). Full preservation proof in `src/CSHRL/Tasks/Verified/TwoState.agda`, with no postulates.

19.2 Delayed Gratification: Depth-Discovery

```
data DGState : Set where
  Start : DGState
  PathA : DGState
  PathB : DGState
  End : DGState

data DGAction : Set where
  GoA : DGAction
  GoB : DGAction
  Wait : DGAction

dg-step : DGState → DGAction → DGState × ℕ
dg-step Start GoA = (PathA , 0)
dg-step Start GoB = (PathB , 0)
dg-step PathA _ = (PathA , 0)
dg-step PathB _ = (End , 1)
dg-step End _    = (End , 1)
dg-step _    _    = (End , 0)

all-actions : List DGAction
all-actions = GoA :: GoB :: []
```

```
open Core DGState DGAction ℕ dg-step _≤_ _⊔_ 0 all-actions
```

At depth 1 both actions yield trace $[0]$—indistinguishable. At depth 2 the traces diverge: `GoA` $\rightarrow [0,0]$, `GoB` $\rightarrow [0,1]$. The conditions again coincide (equal immediate rewards). Full implementation in `src/CSHRL/Tasks/Classic/DelayedGratification.agda`.

A complementary example is **Trap Sparse**: three actions at Start, one of which leads to a terminal reward via a trap state. Unlike Delayed Gratification, the discriminating action has a *different* immediate reward, so depth 1 suffices. Full implementation in `src/CSHRL/Tasks/Classic/TrapSparse.agda`.

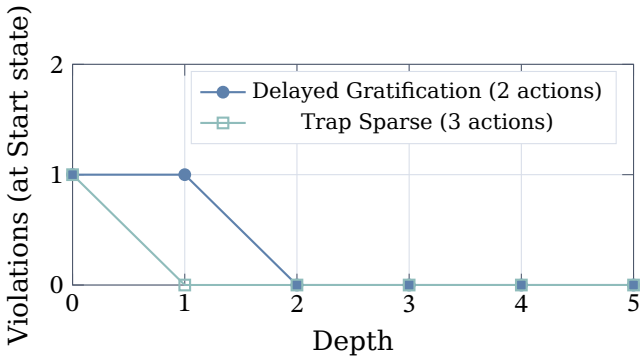


Figure 5: Violations at Start state drop to zero as depth increases. Delayed Gratification requires depth 2 to distinguish equal-immediate-reward paths; Trap Sparse resolves at depth 1. Both are verified in `src/CSHRL/Tasks/Classic/`.

19.3 Key-Door-Treasure: State-Dependent Rankings

A robot must collect a key, unlock a door, then reach treasure.

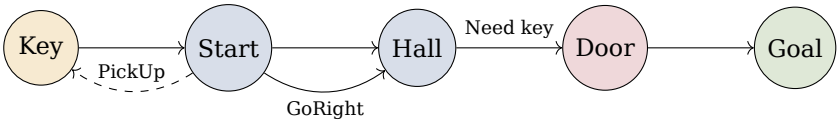


Figure 6: Key–Door–Treasure: the optimal action depends on whether the key is held.

This demonstrates *state-dependent* rankings and the "ranking flip" phenomenon:

State	Without Key	With Key
Start	[PickUp, GoRight, GoLeft]	[GoRight, GoLeft, PickUp]
Hall	[GoLeft, GoRight, Wait]	[GoRight, Wait, GoLeft]

The moment the key is acquired, the ranking inverts—a phase transition encoded directly in the traces rather than via gradual value propagation:

- Without key: `GoRight` $\rightarrow [0,0,0,\ldots]$ (blocked)
- With key: `GoRight` $\rightarrow [0,0,100,100,\ldots]$ (treasure)

This validates that state-dependent rankings instantiate correctly under the `FiniteDeterministicMDP` environment class. The conditions coincide for this environment (uniform step costs). Full implementation in `src/CSHRL/Tasks/Classic/KeyDoor.agda`.

19.4 8-Queens: Scalable Verification

The `Queens8Learning` module validates the full pipeline end-to-end: learning via `materialize-on-path` from the empty board, producing a `PolicyTable` of eight entries. The following extract illustrates the interface; the full verified instantiation is in `src/CSHRL/Tasks/Verified/Queens8.agda`.

```
test-solution : run-policy policy horizon (Ongoing []) horizon
              ≡ C0 :: C4 :: C7 :: C5 :: C2 :: C6 :: C1 :: C3 :: []
test-solution = refl
```

This confirms the materialized policy produces a valid non-attacking 8-Queens placement—verified end-to-end in Agda with `--safe` and zero postulates.

19.5 The Sacrifice Environments: Where the Conditions Separate

The three environments from Section 6—BinarySacrifice, SkillInvestment, and PreparationDilemma—are the verified instantiations that *separate* the two conditions. Each has a machine-checked CoinductiveHomomorphism proof and a machine-checked proof that no CoindHomo can rank the sacrificing actions. These live in `src/CSHRL/Tasks/Verified/BinarySacrifice.agda`, `src/CSHRL/Tasks/Verified/SkillInvestment.agda`, and `src/CSHRL/Tasks/Verified/PreparationDilemma.agda`.

20 Related Work

The framework connects to multiple research traditions but makes a distinct move: replacing scalar aggregation with a coinductive order on infinite futures, and distinguishing strategic evaluation (CoinductiveHomomorphism) from tactical evaluation (CoindHomo).

Approach	Core idea	Relationship
Classical RL [1]	Optimize $\sum_t \gamma^t r_t$ via Bellman equations	CoindHomo *subsumes* this; CoinductiveHomomorphism subsumes successor return (§7)
Policy Gradients [2]	Gradient ascent on expected return	Optimization *method*; our framework defines the *target*
Max-Entropy RL [3]	Entropy bonus for exploration	Full-ranking objective dissolves exploration/exploitation (Remark 3)
AIXI [4]	Universal prior over computable environments	Both are ideals; ours is *structurally* defined
Coalgebra [5]	Infinite behaviors via coinductive definitions	Direct inspiration: $\leq_s$ is a coinductive relation
MDP Homomorphisms [6]	Exploit state/action symmetries for efficiency	Complementary: we use order-preservation, not group automorphisms
Giry Monad [7]	Probability distributions as monad	Direct application: stochastic extension (§11)
CertRL [12]	Coq-verified RL convergence	Complementary: they verify classical RL; we verify coinductive structure

20.1 Lexicographic Multi-Objective RL

The most closely related line of work is **lexicographic multi-objective RL** (LMORL) [8, 9]. LMORL orders *objectives* (different reward signals) lexicographically. Our framework orders *timesteps*: the same reward signal is compared pointwise at $t = 0$, then $t = 1$, then $t = 2$, forever.

- LMORL: "Safety reward first, then efficiency reward."
- Our framework: "At every moment t, the better action's future dominates."

20.2 The Sacrifice Pattern in RL

The sacrifice pattern—accepting immediate cost for long-term benefit—is well-known but typically handled by increasing γ or via reward shaping. Our framework is the first to formally decouple optimality from immediate reward via a coinductive condition. CoinductiveHomomorphism judges actions solely by successor state quality, side-stepping the tension between immediate and future reward that plagues both discounted returns and CoindHomo.

20.3 Constrained and Safe RL

Safe RL [14] treats some objectives as hard constraints rather than soft trade-offs. Constrained MDPs optimize a primary reward while ensuring constraint satisfaction (e.g., collision probability $< \epsilon$).

The framework's relationship to safe RL is complementary:

- **Temporal constraints as dominance:** A safety constraint "never enter state X" translates to: the safe action's trace dominates at every timestep (the unsafe action's trace eventually shows the penalty).
- **No constraint relaxation:** The coinductive order is strict—there is no ϵ-violation. If a's trace dominates b's at every step, the dominance is absolute.

20.4 Preference-Based, Ordinal, and Ranking RL

Preference-based RL (PbRL) [10] learns from pairwise comparisons rather than scalar rewards. This connects to our ranking structure, but the relationship is nuanced:

- PbRL *infers a latent scalar* from comparisons, then optimizes that scalar. Our framework requires only *ordinal structure*—a partial order on outcomes—and compares streams directly without scalar collapse.
- The Core module is parameterized by `_≤ᵣ_ : Reward → Reward → Set`—any partial order suffices. Human preference orderings are a valid instantiation.

Direct Preference Optimization (DPO) [11] bypasses explicit reward modelling by deriving an implicit scalar reward from pairwise preferences via the Bradley–Terry model, then optimizes a policy to maximize it. The pairwise structure of the input is discarded once the implicit reward is extracted. In our framework, by contrast, the pairwise ordering *is* the optimality criterion—preserved coinductively, not collapsed into a scalar.

Deep Ordinal Reinforcement Learning [15] modifies Q-learning for ordinal rewards via thresholded updates, mitigating overestimation in DQN. However, it retains discounts and bootstrapping, unlike the coinductive elimination of γ.

20.5 Coalgebraic Foundations

The coinductive stream order $\leq_s$ is a coalgebraic relation in the sense of Rutten [5]. Coalgebra provides the categorical semantics for infinite behaviors, and our formalization directly instantiates these ideas in Agda's guarded coinduction. The `coinductive` keyword in the $\leq_s$ record is precisely the coalgebraic final coalgebra construction.

MDP homomorphisms [6] exploit state/action symmetries for computational efficiency. The framework uses a different notion of "symmetry": not group-theoretic automorphisms of the state space, but *order-preservation* between rankings and stream dominance.

20.6 Formal Verification of RL

CertRL [12] is a Coq library that machine-checks convergence proofs for value and policy iteration on finite MDPs, using the Giry monad for probabilistic reasoning. While CertRL provides rigorous guarantees for classical tabular RL, our framework extends formal

verification to a different paradigm: coinductive structure preservation rather than scalar convergence. Both projects demonstrate that machine-verified RL theory is feasible; they target complementary foundations.

20.7 What Distinguishes This Work

- **Criterion vs. algorithm:** Classical RL and policy gradients are *algorithms* to optimize a scalar. Our framework defines an *optimality criterion*—a structural property that rankings may or may not satisfy.
- **Temporal structure:** The scalar return $\sum \gamma^t r_t$ erases timing; the coinductive order $\leq_s$ preserves it completely.
- **Pointwise dominance:** Unlike lexicographic MORL (priority among objectives) or average-reward RL (limit of partial sums), the framework requires dominance at *every* timestep.
- **Strategic vs. tactical:** CoinductiveHomomorphism judges actions solely by successor state quality; CoindHomo additionally demands immediate reward alignment. No prior work makes this decomposition.
- **Machine verification:** All results are fully implemented in Agda with type-checked proofs.

21 Future Work

- **Continuous distributions:** The stochastic framework (§11) represents distributions as `List (A × ℕ)`. Extending to measure-theoretic distributions would broaden applicability to continuous-outcome environments.
- **Continuous action spaces:** The framework assumes finite, enumerable actions. Extending to continuous action spaces (e.g., torque values) would require replacing enumeration-based ranking with optimization over compact sets.
- **Partially observable MDPs:** Lift rankings to belief-state distributions with stochastic transitions.
- **Multi-agent settings:** Rankings over joint action spaces with game-theoretic equilibrium conditions.
- **Program synthesis as policy representation:** The full-ranking objective (Remark 3) naturally extends to synthesized programs: a program satisfying all pairwise ranking constraints on observed states encodes sufficient environment structure to generalize to unseen states.

22 Conclusion

We have presented a framework that redefines optimality as *structure preservation* rather than scalar maximization. Two coinductive conditions capture two complementary notions of optimality:

- **CoinductiveHomomorphism** evaluates actions by the quality of the states they lead to—successor state quality. It is the strategic condition: an action that sacrifices immediate reward to reach a superior successor is correctly ranked.
- **CoindHomo** additionally requires that immediate transition rewards respect the ranking. It is the tactical condition: the better-ranked action must win both the immediate payoff and the long-term future.

The decomposition theorem establishes their precise relationship: CoindHomo equals CoinductiveHomomorphism plus immediate reward compatibility. For the vast majority of standard environments, these conditions coincide; three verified counterexamples demonstrate the strict gap.

The framework is supported by machine-verified results in Agda:

1. **Subsumption:** Both conditions imply classical argmax optimality for any finite horizon.
2. **Monotonic learning:** Violation-correction converges in at most $|S| \times \binom{|A|}{2}$ swaps.
3. **Instant adaptation:** Action unavailability requires $O(1)$ lookup, not full recomputation.
4. **Stochastic extension:** The framework lifts to probabilistic environments via the Giry monad, with lexicographic coinductive comparison.
5. **Distributional hierarchy:** FOSD and the SD[k] chain (§12–13) give parameterized verification strength for progressively broader utility function classes.
6. **Compositional algebra:** Verified rankings compose via product, sum, convolution, and scaling.
7. **State abstraction:** Rankings verified on finite abstract systems lift to arbitrary concrete spaces.
8. **Scalable verification:** The Environment Class architecture provides automatic trace-to-stream bridges; 8-Queens achieves a full CoindHomo for 8^8 candidate placements with zero postulates.

This document is itself the proof: compiling it type-checks all embedded code while producing this PDF.

23 Reproducibility and Build

- **Tested toolchain:** Agda 2.7.x, Agda StdLib 2.0, LuaLaTeX.
- **Build commands:** In `literate/`, run `make successor` (or `agda --latex CSHRL-Successor.lagda.tex` followed by `lualatex`).
- **Fonts:** The document uses `fontspec` and Unicode math fonts (DejaVu, STIX Two Math).

References

[1] Sutton, R. S. and Barto, A. G. (2018). *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press.

[2] Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4):229–256.

[3] Haarnoja, T., Zhou, A., Abbeel, P., and Levine, S. (2018). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of ICML*, pp. 1861–1870.

[4] Hutter, M. (2005). *Universal Artificial Intelligence: Sequential Decisions Based on Algorithmic Probability*. Springer.

[5] Rutten, J. J. M. M. (2000). Universal coalgebra: A theory of systems. *Theoretical Computer Science*, 249(1):3–80.

[6] van der Pol, E., Worrall, D., van Hoof, H., Oliehoek, F., and Welling, M. (2020). MDP homomorphic networks: Group symmetries in reinforcement learning. In *NeurIPS*.

[7] Giry, M. (1982). A categorical approach to probability theory. In *Categorical Aspects of Topology and Analysis*, LNCS vol. 915, pp. 68–85. Springer.

[8] Skalse, J., Hammond, L., Griffin, C., and Abate, A. (2022). Lexicographic multi-objective reinforcement learning. In *IJCAI*, pp. 3430–3436.

[9] Tercan, H. and Prabhu, S. (2024). Thresholded lexicographic ordered multiobjective reinforcement learning. In *ECAI*.

[10] Wirth, C., Akrour, R., Neumann, G., and Fürnkranz, J. (2017). A survey of preference-based reinforcement learning methods. *JMLR*, 18(136):1–46.

[11] Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C. D., and Finn, C. (2023). Direct preference optimization: Your language model is secretly a reward model. In *NeurIPS*.

[12] Vajjha, K., Shinnar, A., Trager, B., Pestun, V., and Fulton, N. (2021). CertRL: Formalizing convergence proofs for value and policy iteration in Coq. In *CPP '21: Certified Programs and Proofs*, pp. 18–31. ACM.

[13] Li, L., Walsh, T. J., and Littman, M. L. (2006). Towards a unified theory of state abstraction for MDPs. In *ISAIM*.

[14] García, J. and Fernández, F. (2015). A comprehensive survey on safe reinforcement learning. *JMLR*, 16(42):1437–1480.

[15] Zap, A., Chaudhuri, S., and Topcu, U. (2019). Deep ordinal reinforcement learning. In *AAMAS*, pp. 1838–1846.

[16] Bellman, R. (1957). *Dynamic Programming*. Princeton University Press.

[17] Watkins, C. J. C. H. and Dayan, P. (1992). Q-learning. *Machine Learning*, 8(3–4):279–292.

[18] Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533.